

```
/* ===== PICO W
— ROVER 2 Receives TCP alerts from ESP32-CAM (RoverCAM_AP_R2) Forwards them
over UART1 to ESP8266 LoRa transmitter
```

WiFi: connects to "RoverCAM_AP_R2" / "rover1234" ESP32-CAM IP: 192.168.4.1 Port: 5000

UART1: GP4(TX) → ESP8266 GPIO3(RX) @ 115200 GP5(RX) ← ESP8266 GPIO1(TX) @ 115200

Messages forwarded: <MSG|R2|HUMAN_DETECTED> <MSG|R2|FIRE_DETECTED>
===== */

```
#include "pico/stdlib.h" #include "pico/cyw43_arch.h" #include "lwip/tcp.h" #include
"hardware/uart.h" #include <stdio.h> #include <string.h>
```

```
// ----- // WiFi — connects to ESP32-CAM AP
— ROVER 2 // ----- #define WIFI_SSID
"RoverCAM_AP_R2" // ← R2 #define WIFI_PASSWORD "rover1234" #define
CAM_SERVER_IP "192.168.4.1" #define CAM_SERVER_PORT 5000
```

```
// ----- // UART1 to ESP8266 // -----
----- #define UART_ID uart1 #define UART_TX 4 // GP4 #define
UART_RX 5 // GP5 #define UART_BAUD 115200
```

```
// ----- // Buffer // -----
----- #define MAX_BUF 200
```

```
static char rxBuf[MAX_BUF]; static int rxLen = 0;
```

```
// ----- // UART init // -----
----- void uart_init_r2() { uart_init(UART_ID, UART_BAUD);
gpio_set_function(UART_TX, GPIO_FUNC_UART); gpio_set_function(UART_RX,
GPIO_FUNC_UART); printf("[UART] UART1 ready — GP4(TX) GP5(RX) @ %d\n",
UART_BAUD); }
```

```
// ----- // Forward message to ESP8266 via
UART // ----- void forward_to_esp8266(const
char* msg) { uart_puts(UART_ID, msg); uart_puts(UART_ID, "\n"); printf("[UART]
Forwarded to ESP8266 R2 → %s\n", msg); }
```

```
// ----- // TCP receive callback // -----
----- static err_t tcp_recv_cb(void *arg, struct tcp_pcb
```

```
*tpcb, struct pbuf *p, err_t err) { if (!p) { printf("[TCP] Connection closed by ESP32-CAM R2\n"); tcp_close(tpcb); return ERR_OK; }
```

```
char *data = (char*)p->payload;  
uint16_t len = p->len;
```

```
for (uint16_t i = 0; i < len; i++) {  
    char c = data[i];
```

```
    if (c == '\n') {  
        rxBuf[rxLen] = '\0';  
        // Trim \r if present  
        if (rxLen > 0 && rxBuf[rxLen - 1] == '\r') {  
            rxBuf[--rxLen] = '\0';  
        }  
        if (rxLen > 0) {  
            printf("[TCP] Received from CAM R2: %s\n", rxBuf);  
            // Only forward detection alerts, not handshake  
            if (strstr(rxBuf, "R2|") != NULL) { // ← R2  
                forward_to_esp8266(rxBuf);  
            }  
        }  
        rxLen = 0;  
    } else if (c != '\r') {  
        if (rxLen < MAX_BUF - 1) {  
            rxBuf[rxLen++] = c;  
        }  
    }  
}
```

```
tcp_recved(tpcb, p->tot_len);  
pbuf_free(p);  
return ERR_OK;
```

```
}
```

```
// ----- // TCP connected callback // -----  
----- static err_t tcp_connected_cb(void *arg, struct  
tcp_pcb *tpcb, err_t err) { if (err != ERR_OK) { printf("[TCP] Connection to CAM R2  
failed: %d\n", err); return err; } printf("[TCP] Connected to ESP32-CAM R2!\n");  
tcp_recv(tpcb, tcp_recv_cb); return ERR_OK; }
```

```

// ----- // Connect to ESP32-CAM TCP server //
----- struct tcp_pcb* connect_to_cam()
{ ip_addr_t server_ip; ip4addr_aton(CAM_SERVER_IP, &server_ip);

struct tcp_pcb *pcb = tcp_new();
if (!pcb) {
    printf("[TCP] tcp_new() failed\n");
    return NULL;
}

err_t err = tcp_connect(pcb, &server_ip, CAM_SERVER_PORT,
tcp_connected_cb);
if (err != ERR_OK) {
    printf("[TCP] tcp_connect() failed: %d\n", err);
    tcp_abort(pcb);
    return NULL;
}

printf("[TCP] Connecting to ESP32-CAM R2 at %s:%d ... \n",
    CAM_SERVER_IP, CAM_SERVER_PORT);
return pcb;

}

// ----- // main() // -----
----- int main() { stdio_init_all(); sleep_ms(2000);

printf("\n===== \n");
printf("  Pico W – Rover 2\n");
printf("  TCP Client → ESP32-CAM R2\n");
printf("  UART1 → ESP8266 LoRa TX R2\n");
printf("===== \n\n");

uart_init_r2();

// Init WiFi
if (cyw43_arch_init()) {
    printf("[WiFi] Init failed!\n");
    return 1;
}

cyw43_arch_enable_sta_mode();

```

```

printf("[WiFi] Connecting to %s ...\n", WIFI_SSID);

int retry = 0;
while (cyw43_arch_wifi_connect_timeout_ms(
    WIFI_SSID, WIFI_PASSWORD,
    CYW43_AUTH_WPA2_AES_PSK, 10000) != 0) {
    printf("[WiFi] Retry %d ...\n", ++retry);
    sleep_ms(2000);
}

printf("[WiFi] Connected to RoverCAM_AP_R2!\n");
printf("[WiFi] IP: %s\n",
    ip4addr_ntoa(netif_ip4_addr(netif_list)));

// Connect to ESP32-CAM TCP server
struct tcp_pcb *cam_pcb = NULL;
while (!cam_pcb) {
    cam_pcb = connect_to_cam();
    if (!cam_pcb) {
        printf("[TCP] Retrying in 3s ...\n");
        sleep_ms(3000);
    }
}

// Main loop – lwIP handles everything via callbacks
printf("[Main] Running – waiting for detections from CAM
R2...\n\n");
while (true) {
    cyw43_arch_poll();
    sleep_ms(1);
}

cyw43_arch_deinit();
return 0;

}

```